

Rekursion

Rekursion är ett begrepp som brukar förknippas med funktionell programmering. Trots att C++ är ett imperativt språk kan man även här använda rekursion för att lösa olika problem. Rekursion innebär som bekant att funktioner kan anropa sig själv. Rekursion är inte alltid det mest optimala förfarandet i C++, men i vissa fall är det väldigt praktiskt, och kan t.o.m. vara effektivare än iterativ kod. T.ex. vid hantering av *träd* av olika typ är rekursion en ovärderlig resurs. Ett standardexempel på rekursion torde vara hur $n!$ (n-fakultet) räknas ut rekursivt:

```
#include <iostream>

unsigned int Fakultet (unsigned int N) {
    if ( N == 0 ) {
        // basfallet nått
        return 1;
    }

    return N * Fakultet ( N - 1 );
}

int main () {
    unsigned int Tal;
    cout << "Ge in ett tal: ";
    cin >> Tal;

    // beräkna fakulteten och skriv ut svaret
    cout << Tal << "! = " << Fakultet ( Tal ) << endl;
}
```

Man bör dock se upp för oändlig eller alltför djup rekursion, eftersom det leder till problem med programmets stack, vilket i sin tur leder till problem i exekveringen. Ett annat exempel på rekursiv programmering är beräkning av *Fibonacci-tal*. Fibonacci-talet $\text{fib}(n)$ definieras som $\text{fib}(n-1) + \text{fib}(n-2)$, där $\text{fib}(0) = 0$ och $\text{fib}(1) = 1$. Vi kan nu använda denna definition för att skapa en enkel implementation.

```
#include <iostream>

unsigned int fib (unsigned int N) {
    if ( N == 0 ) {
        // basfall 1 nått
        return 0;
    }

    if ( N == 1 ) {
        // basfall 2 nått
        return 1;
    }

    // allmänt fall, använd rekursion
    return fib ( N - 1 ) + fib ( N - 2 );
}

int main () {
    unsigned int Tal;
```

```
cout << "Ge in ett tal: ";
cin >> Tal;

// beräkna fakulteten och skriv ut svaret
cout << Tal << "! = " << fib ( Tal ) << endl;
}
```

[Förutgående](#)

Omfattning (scoping)

[Hem](#)[Upp](#)[Nästa](#)

Funktioner som parametrar